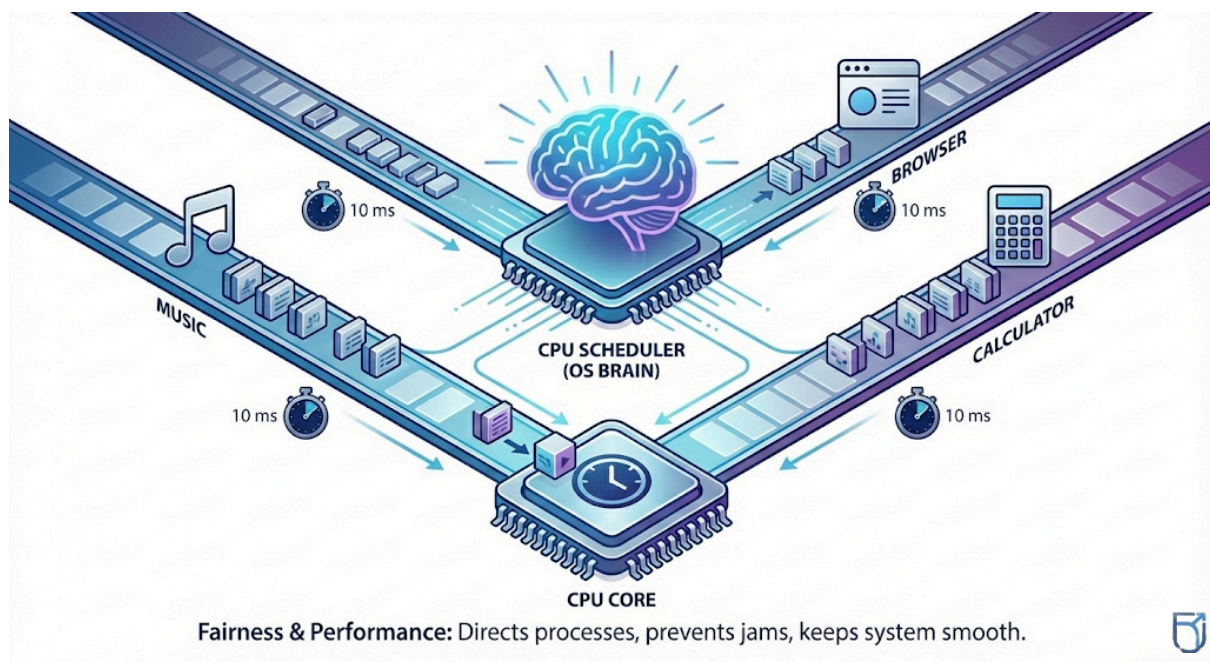


Lecture 06 NOTES– CPU Scheduling Fundamentals

What is CPU Scheduling?

Think of this as the brain of the Operating System making decisions on who gets to use the processor.

- **OS decision of which process runs next:** Since a CPU core can only run one task at a time, the scheduler decides which program (process) gets the CPU and which ones have to wait in line.
- **Allocation of CPU time:** It's responsible for slicing up time. For example, it might give 10 milliseconds to your music player, then 10 milliseconds to your web browser, switching back and forth so quickly that everything looks like it's running at once.

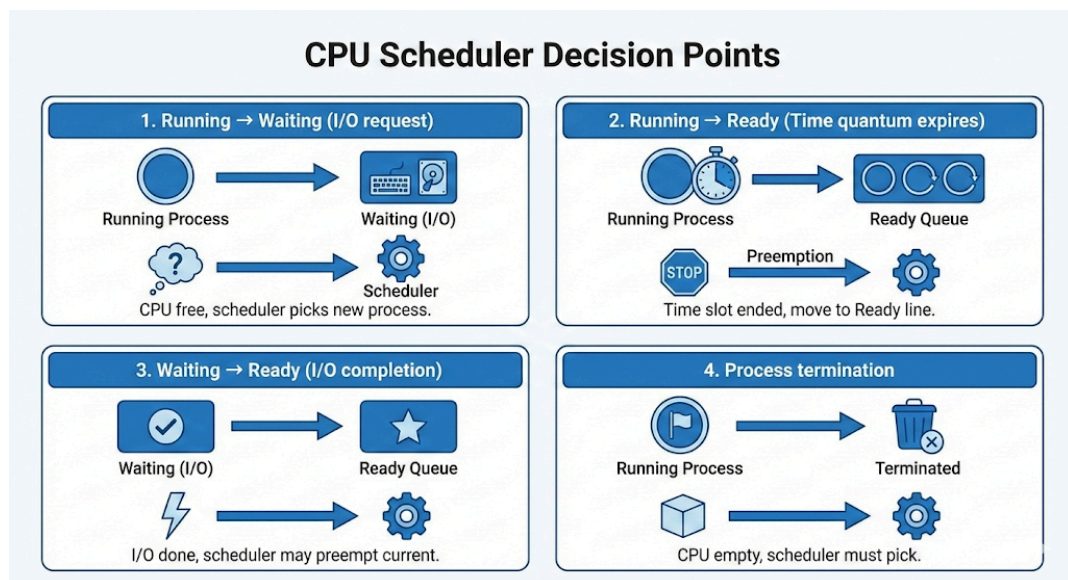


- **Fairness and performance goals:**
 - **Fairness:** It makes sure no single program hogs the CPU forever (starvation) so that every app gets a chance to run.
 - **Performance:** It tries to keep the CPU busy 100% of the time. If one program is waiting for a file download, the scheduler swaps in another program so the CPU isn't sitting idle.
- **Acts as OS traffic controller:** Just like a traffic cop at a busy intersection directs cars to prevent jams and keep traffic flowing, the scheduler directs processes to keep the system running smoothly without freezing.

When CPU Scheduling Happens

The scheduler has to wake up and make a decision in four specific situations:

1. **Running → Waiting (I/O request):**
 - **What happens:** A program voluntarily gives up the CPU because it can't proceed until an external event happens (like waiting for you to press a key or waiting for a file to read from the hard drive).
 - **Result:** The CPU is free, so the scheduler picks a new process.
2. **Running → Ready (Time quantum expires):**
 - **What happens:** The program was running, but its assigned time slot (time quantum) ran out, or an interrupt occurred (like a timer ticking).
 - **Result:** The OS forces the program to pause and go back to the "ready" line so someone else can have a turn. This is called **preemption**.



3. **Waiting → Ready (I/O completion):**
 - **What happens:** A program that was waiting for something (like a download) finally gets what it needed. It moves back to the "ready" line.
 - **Result:** The scheduler *might* decide to stop the current running process immediately to let this newly ready process run (especially if it has higher priority).
4. **Process termination:**
 - **What happens:** A program finishes its job or crashes and closes.
 - **Result:** The CPU is now completely empty, so the scheduler **must** pick a new process from the ready queue immediately.

Scheduling Goals & Challenges

- **Cannot optimize all goals simultaneously:** It is impossible to make everyone happy. For example, if you want to make the system super efficient

(high throughput), you might have to delay some tasks for a long time, which hurts responsiveness.

- **Trade-offs between performance metrics:**
 - **Fairness vs. Speed:** Being perfectly fair means switching often so everyone gets a turn, but switching takes time, which slows down the overall work speed.
 - **Response vs. Throughput:** To make apps feel snappy (response), you need to pause heavy background calculations often, which reduces the total amount of heavy work done (throughput).
- **Different systems → different priorities:**
 - **Mainframe/Server:** Cares about crunching numbers fast (**Throughput**).
 - **Your Laptop/Phone:** Cares about not lagging when you click a button (**Response Time**).
 - **Car Airbag System (Real-time):** Cares about guaranteed deadlines; speed doesn't matter if it's late.

Scheduling Criteria (Performance Metrics)

These are the "scorecards" we use to grade how good a scheduling algorithm is.



- **CPU Utilization:** How busy is the brain? (We want this **High**).
- **Throughput:** How many jobs are finished per second? (We want this **High**).
- **Turnaround Time:** How long does it take from the moment you hit "Enter" until the program completely finishes? (We want this **Low**).
- **Waiting Time:** How much time did the process spend sitting in the "Ready Queue" doing nothing? (We want this **Low**).
- **Response Time:** How long does it take from the moment you hit "Enter" until the *first* thing appears on the screen? (We want this **Low** - critical for games and UI).

Throughput & Turnaround Time

- **Throughput (Work Rate):** The number of processes completed per second/minute. (Higher is better for system efficiency).
- **Turnaround Time (Total Duration):** The total time taken from submitting a process to its final completion.
 - *Formula:* Completion Time - Arrival Time.
- **The Trade-off:** Strategies that maximize **Throughput** (completing many small jobs) often hurt **Turnaround Time** for larger jobs.

Waiting Time & Response Time

- **Waiting Time (Queue Time):** Total time a process spends sitting in the Ready Queue waiting for the CPU. (It does not include time actually running or doing I/O).
- **Response Time (Reaction Speed):** The time from submission until the process gets the CPU for the **first** time.
- **Key Insight: Response Time** is the most critical metric for interactive systems (like PCs/Phones) because users notice delay immediately.

System Types & Goals

- **Batch Systems (e.g., Payroll, Weather simulation):** Focus on **Throughput** and CPU utilization. No human is waiting, so response time doesn't matter.
- **Interactive Systems (e.g., Your Laptop, Phone):** Focus on **Response Time** and **Fairness**. The system must feel snappy to the user.
- **Real-Time Systems (e.g., Airbags, Robots):** Focus on **Deadlines**. A task must finish by a specific time, or the system fails.

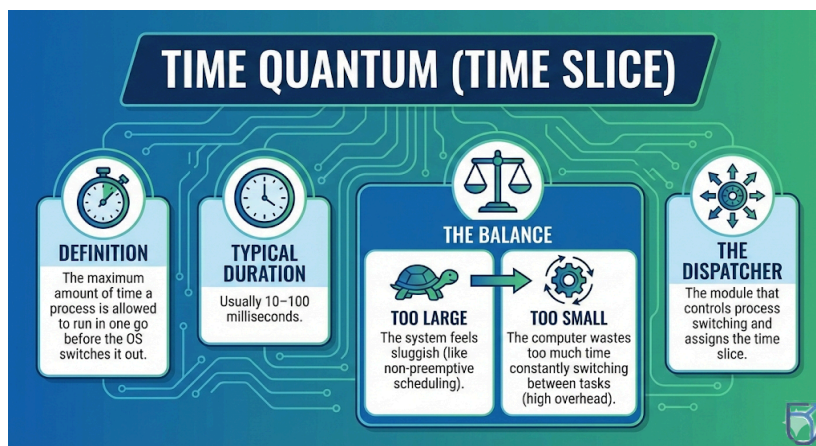
Difference: Preemptive vs. Non-Preemptive Scheduling

Feature	Preemptive Scheduling (Interruptible)	Non-Preemptive Scheduling (Greedy)
Basic Concept	The OS can forcibly pause a running process to run a more important one	Once a process starts, it keeps the CPU until it finishes or waits for I/O
Control	OS controls the timeline (interrupts via timer)	Process controls the timeline (voluntary release)

Responsiveness	High: Great for interactive apps (no freezing)	Low: One slow program can freeze the whole system
Overhead	High: Requires frequent context switching (uses more CPU power)	Low: Fewer switches means less overhead
Best For	Modern Systems (Windows, macOS, Android)	Batch jobs or very simple hardware

Time Quantum (Time Slice)

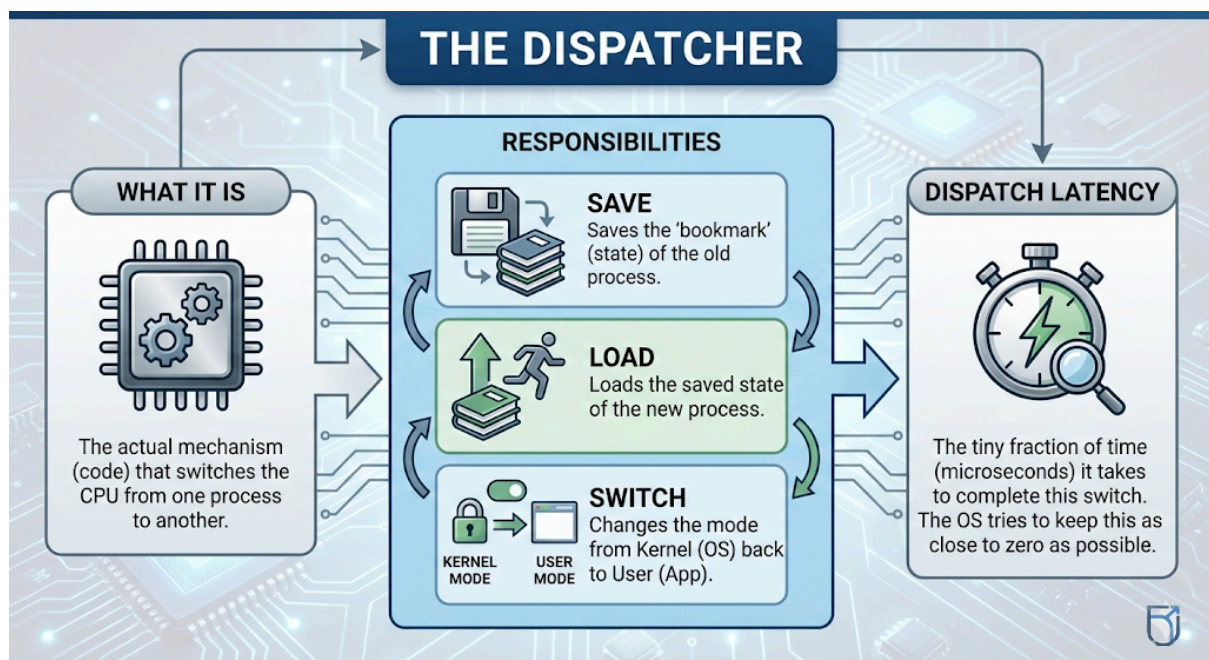
- **Definition:** The maximum amount of time a process is allowed to run in one go before the OS switches it out.
- **Typical Duration:** Usually **10–100 milliseconds**.



- **The Balance:**
 - **Too Large:** The system feels sluggish (like non-preemptive scheduling).
 - **Too Small:** The computer wastes too much time constantly switching between tasks (high overhead).

The Dispatcher

- **What it is:** The actual mechanism (code) that switches the CPU from one process to another.



- **Responsibilities:**
 1. **Save:** Saves the "bookmark" (state) of the old process.
 2. **Load:** Loads the saved state of the new process.
 3. **Switch:** Changes the mode from Kernel (OS) back to User (App).
- **Dispatch Latency:** The tiny fraction of time (microseconds) it takes to complete this switch. The OS tries to keep this as close to zero as possible.

Scheduling Queues

Processes move between different lines (queues) depending on what they need.

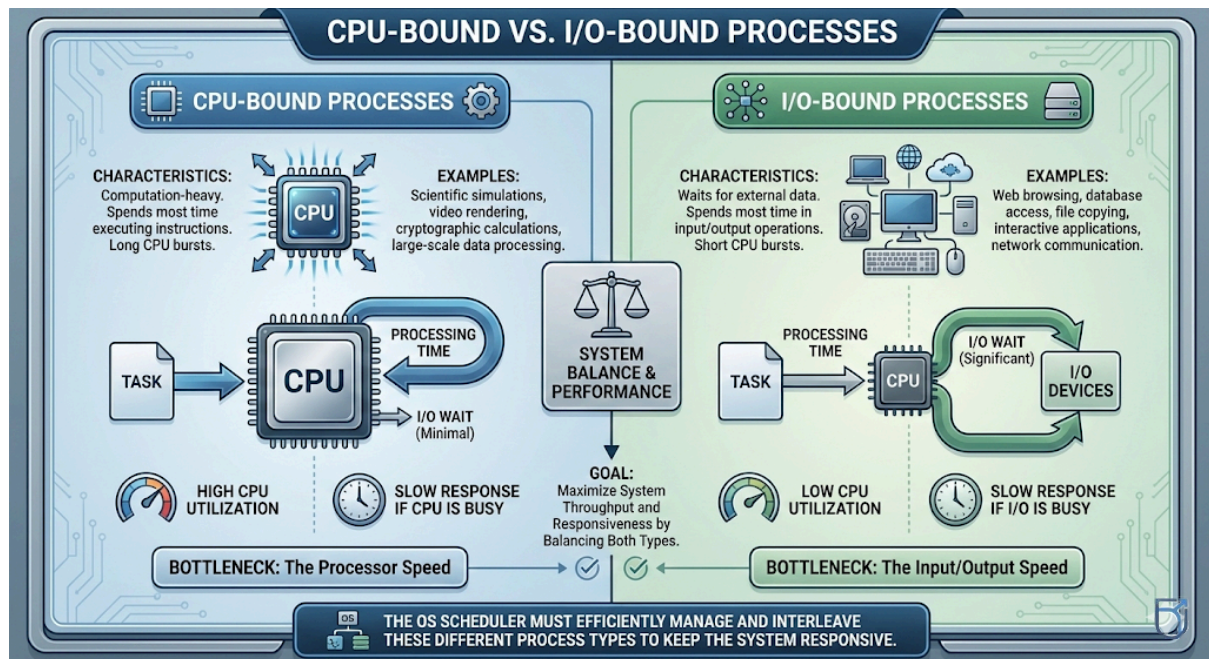
- **Job Queue:** This is the "Entry Hall." It tracks **every** process currently in the system, whether it's running, waiting, or ready.
- **Ready Queue (The Critical Line):**
 - This is the most important queue.
 - It holds processes that are in memory and **ready to run** immediately. They are just waiting for the CPU to become free.
- **I/O Queues:**
 - These are separate lines for specific devices (like the disk, network, or printer).
 - If a process needs to print a file, it moves from the Ready Queue to the Printer I/O Queue to wait its turn.

CPU-Bound vs. I/O-Bound Processes

Processes usually fall into one of two "personalities" based on how they spend their time.

A. CPU-Bound (The "Thinkers")

- **Behavior:** They spend most of their time doing heavy calculations.
- **Characteristics:** They have very **long CPU bursts** (doing work for a long time without pausing) and rarely ask for I/O.
- **Examples:** Video editing, compiling code, gaming graphics.
- **Goal:** The scheduler gives them long time slots to get work done (Throughput).



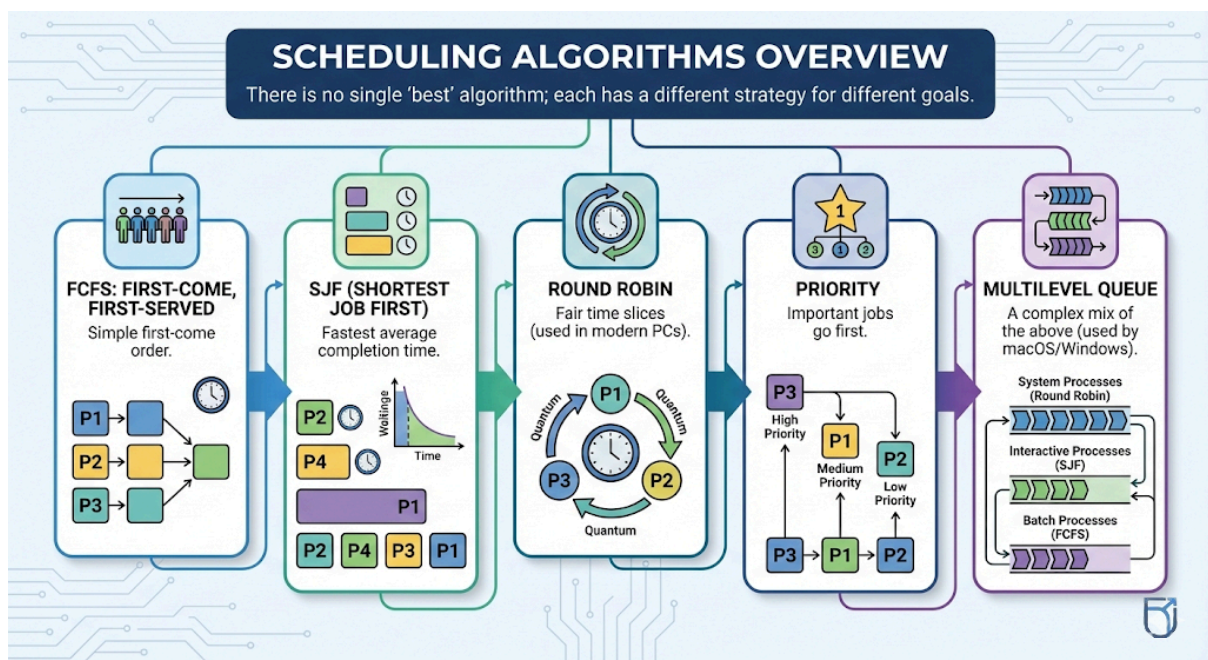
B. I/O-Bound (The "Waiters")

- **Behavior:** They spend most of their time waiting for user input or data downloads.
- **Characteristics:** They have very **short CPU bursts** (using the CPU for just a millisecond) but use it frequently.
- **Examples:** Text editors, web browsers, chat apps.
- **Goal:** The scheduler gives them **high priority** so they respond instantly when you type.

Scheduling Algorithms Overview

There is no single "best" algorithm; each has a different strategy for different goals.

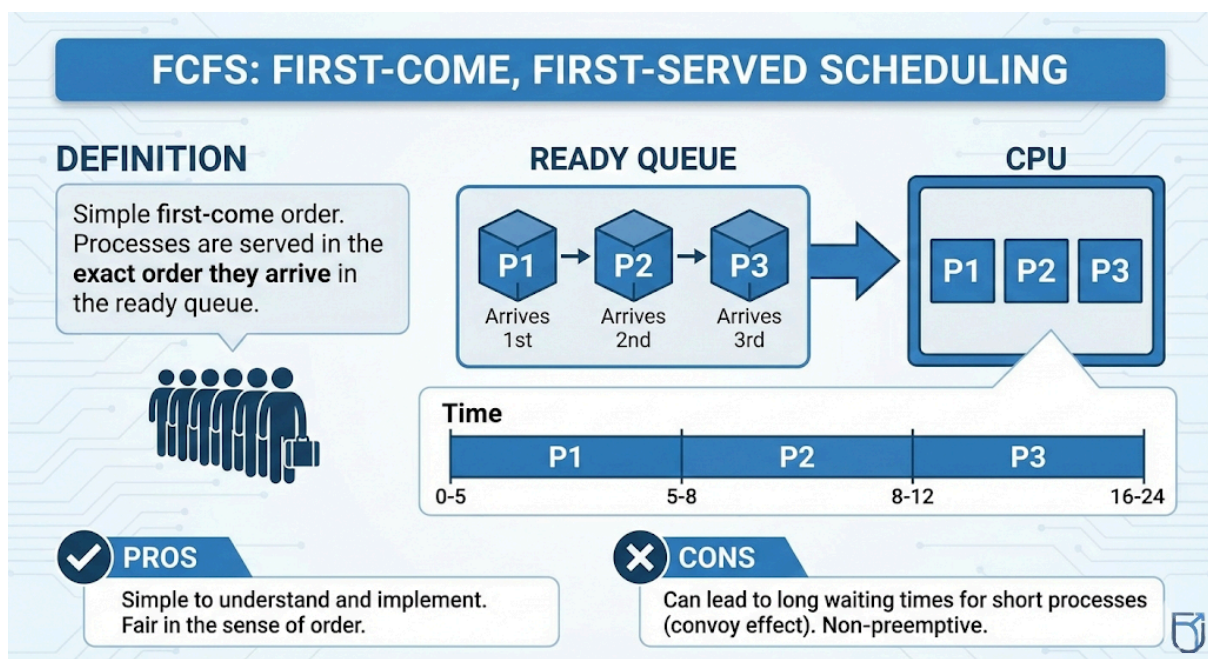
- **FCFS:** Simple first-come order.
- **SJF (Shortest Job First):** Fastest average completion time.
- **Round Robin:** Fair time slices (used in modern PCs).
- **Priority:** Important jobs go first.
- **Multilevel Queue:** A complex mix of the above (used by macOS/Windows).



FCFS (First-Come, First-Served)

This is the simplest algorithm, functioning exactly like a grocery store checkout line.

- **How it works:** The process that arrives first gets the CPU first and keeps it until it is completely finished.



- **Pros:**
 - **Simple:** Very easy to write the code for.
 - **Fair:** Everyone gets served in the exact order they arrived.
- **Cons:**
 - **The Convoy Effect:** If one huge CPU-bound process (the "big truck") gets in front, all the small, fast I/O processes (the "fast cars") are stuck waiting behind it.

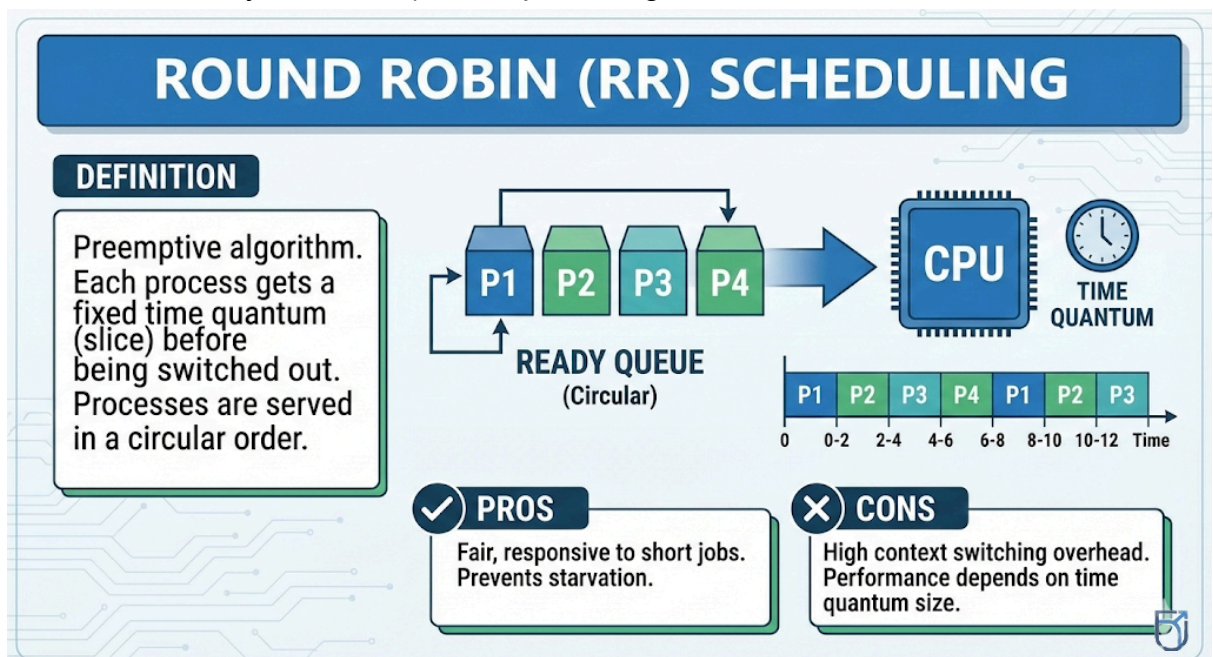
- **Poor Waiting Time:** It usually results in a high average waiting time because short jobs have to wait for long jobs to finish.

SJF (Shortest Job First)

- **The Logic:** The scheduler looks at all waiting processes and picks the one that needs the **least amount of time** to finish its current task (burst).
- **The Superpower:** It is mathematically proven to give the **minimum average waiting time**. By clearing small jobs quickly, fewer people are left waiting in line.
- **The Catch:**
 - **Prediction:** The OS doesn't actually know how long a job will take. It has to guess based on the past (e.g., "This job usually takes 5ms").
 - **Starvation:** If short jobs keep arriving, a long job (like a video render) might sit at the back of the line forever and never run.

Round Robin (RR)

- **The Logic:** This is designed for fairness. The OS sets a timer (called a **Time Quantum**, usually 10-100ms). Each process gets that much time on the CPU.

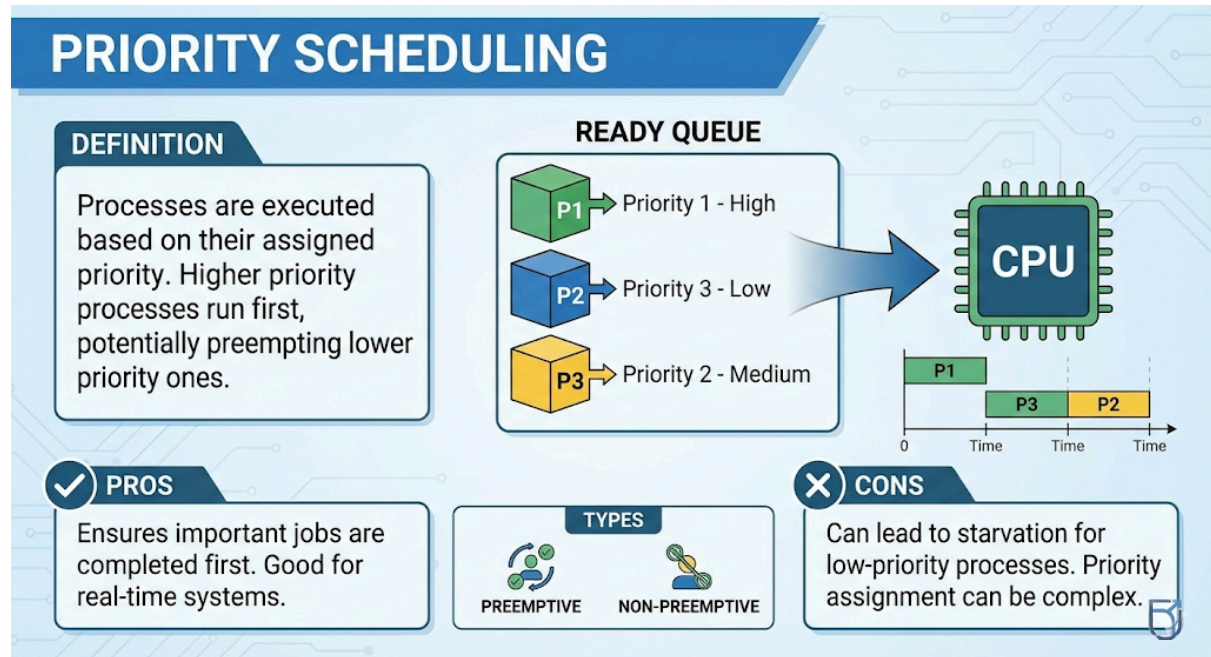


- **The Balance:**
 - **Too Long:** If the timer is too long (e.g., 10 seconds), it acts just like FCFS, and the system feels laggy.
 - **Too Short:** If the timer is too short (e.g., 1 microsecond), the CPU wastes all its energy switching between tasks rather than doing actual work.

Priority Scheduling

- **The Logic:** Not all jobs are equal. The OS assigns a rank to each process. The CPU is always given to the process with the **highest priority**.

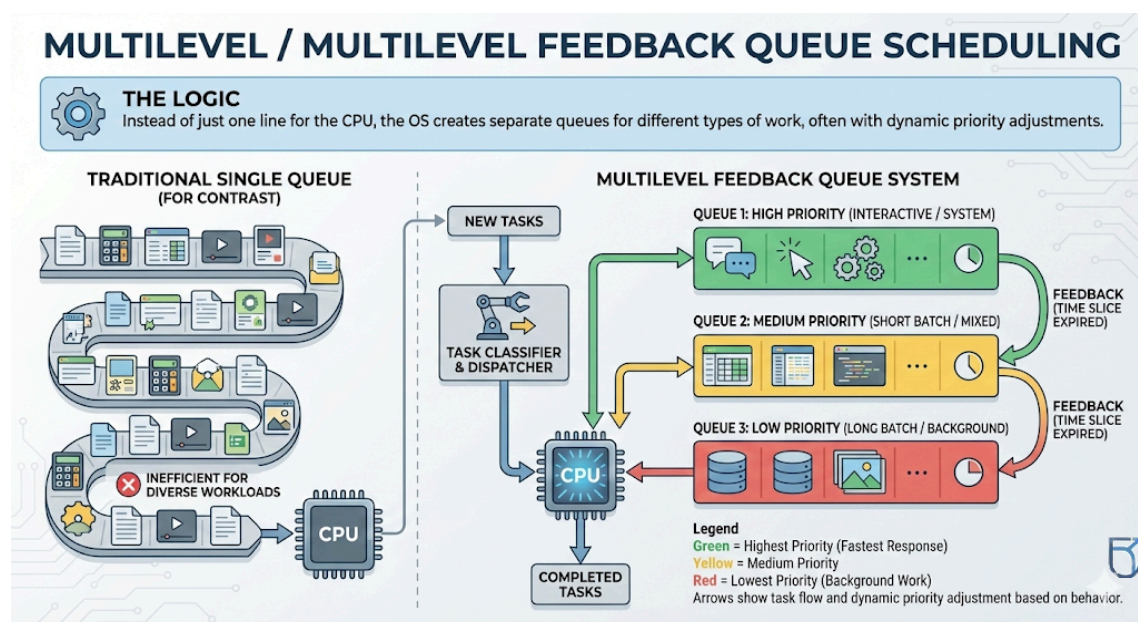
- **Real-World Example:** On your phone, the app you are currently touching (Foreground) has higher priority than an app updating in the background.



- **The Problem (Starvation):** A low-priority job might wait indefinitely if high-priority jobs keep showing up.
- **The Fix (Aging):** As a low-priority job waits, the OS slowly increases its priority over time. Eventually, it becomes "high priority" enough to run.

Multilevel / Multilevel Feedback Queue

- **The Logic:** Instead of just one line for the CPU, the OS creates **separate queues** for different types of work.
 - *Queue 1 (High Priority):* Interactive apps (needs Round Robin for quick response).
 - *Queue 2 (Low Priority):* Batch jobs (needs FCFS for efficiency).



- **Feedback:** "Feedback" means processes can move between queues. If a process uses too much CPU, it is demoted to a lower queue. If it waits too long, it is promoted.
- **Usage:** This is the standard for modern systems like **macOS** (which has 128 priority bands) and Windows.

Context Switching Cost

- **What it is:** The "cleanup and setup" time when changing processes. It typically takes **1-5 microseconds**.
- **Why it's expensive:** It isn't just saving data; it "pollutes" the cache. The CPU's high-speed memory is full of data for the *old* process, which is useless for the *new* one, forcing the CPU to fetch data from slower RAM.
- **Goal:** The OS tries to keep this "wasted time" to less than 1% of total CPU time.

The Illusion of Parallelism

- **The Trick:** Your CPU (even with 4 cores) creates the illusion that 100+ apps are running at once.
- **How:** It switches between them so rapidly (every 10-100ms) that human eyes cannot perceive the gap.
- **Efficiency:** While one app waits for a disk file (I/O), the CPU instantly switches to another app to keep working, maximizing hardware usage.

Why CPU Scheduling Matters

- **User Experience:** Bad scheduling leads to "frozen" screens, audio skipping, or laggy typing.
- **Efficiency:** Good scheduling ensures the expensive CPU you paid for isn't sitting idle while work is piling up.
- **Battery Life:** On mobile devices, efficient scheduling allows the CPU to finish work fast and go back to sleep, saving power.